Rapport 1 : Analyse Théorique et Mécanismes de l'Apprentissage Fédéré

Mohamed Amine Chemkhi
Master 2 IoT – Projet *Fed-SemanticIoT*
Superviseure : Mme Nancy Awad

Novembre 2026

Ce document répond aux questions méthodologiques soulevées lors de la revue de projet concernant le fonctionnement interne de l'apprentissage fédéré (*Federated Learning* – FL), la sélection des clients, les stratégies d'agrégation et les protocoles d'évaluation.

---

1 Quand et pourquoi choisit-on qu'un client participe à un round ?

Dans un déploiement réel (flotte d'équipements IIoT ou terminaux mobiles), le serveur central ne sélectionne qu'une fraction $C \in (0, 1]$ des K clients disponibles à chaque round t. Cette sélection repose sur deux familles de critères :

1.1 Critères matériels et systèmes (Éligibilité)

Un client ne peut être sélectionné que s'il remplit des conditions opérationnelles strictes afin d'éviter l'effet **straggler** (un client lent qui retarde la synchronisation globale du serveur) :

- **Disponibilité énergétique et CPU/RAM :** L'appareil embarqué ne doit pas être en pleine charge sur une tâche industrielle critique temps-réel.
- **Qualité du lien réseau :** Bande passante suffisante et latence stable pour transmettre les matrices de poids sans coupure (*timeout*).

1.2 Critères algorithmiques (Stratégie d'échantillonnage)

- **Échantillonnage aléatoire uniforme (Standard FedAvg) :** Le serveur tire au sort une fraction C parmi les clients éligibles (`np.random.choice`).
- **Sélection active basée sur l'utilité (ex: Oort, Power-of-Choice) :** Le serveur sélectionne prioritairement les clients dont l'erreur locale (*Loss* $F_k(w)$) est la plus élevée. Un client qui fait beaucoup d'erreurs possède des données que le modèle global ne maîtrise pas encore ; sa participation apporte donc un gain d'information maximal.

2 Relation entre la distribution des données, la participation au round et la contribution à l'agrégation

La distribution des données locales D_k impacte directement la convergence selon deux axes :

2.1 Le déséquilibre de volume (Quantity Skew)

Si le Client 1 possède $n_1 = 10\,000$ échantillons et le Client 2 seulement $n_2 = 100$ échantillons, leur contribution lors de l'agrégation FedAvg est pondérée par le ratio $\frac{n_k}{n_{S_t}}$, où S_t est l'ensemble des clients sélectionnés au round t et $n_{S_t} = \sum_{k \in S_t} n_k$:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{n_{S_t}} w_k^{t+1} \tag{1}$$

Conséquence : Le Client 1 domine mathématiquement la mise à jour (son poids dans la moyenne est 100 fois supérieur). Si les petits clients participent rarement, leurs spécificités locales sont écrasées.

2.2 Le déséquilibre statistique (Non-IID : Label Skew / Feature Skew)

Lorsque chaque client ne voit qu'un sous-ensemble de classes (ex: dans notre test MNIST, si le Client 1 n'a que les chiffres `0–4` et le Client 2 les chiffres `5–9`), les gradients locaux $\nabla F_k(w)$ pointent vers des minima locaux opposés :

$$w_k^{t+1} = w_t - \eta \nabla F_k(w_t) \tag{2}$$

Impact de la participation : Si un sous-groupe de clients ayant la même distribution participe plusieurs rounds de suite sans les autres, le modèle global subit un **Client Drift** (dérive des poids) et un phénomène d'**oubli catastrophique** (*catastrophic forgetting*) des classes détenues par les clients absents.

3 Comment le serveur connaît-il ou décide-t-il de la contribution de chaque client ?

Il existe deux approches selon le niveau de confiance du réseau :

1. **Approche déclarative (Confiance coopérative – FedAvg classique) :**
   À la fin de l'entraînement local, le client renvoie un tuple contenant ses poids et la taille de son dataset : (w_k^{t+1}, n_k). Dans le framework **Flower**, cela correspond au retour de la fonction `fit()` : `return self.get_parameters(), len(self.x_local), {}`. Le serveur utilise directement cet entier `len(self.x_local)` pour calculer le coefficient $\frac{n_k}{n}$.
2. **Approche évaluative par le serveur (Zéro-Confiance / Défense Byzantine) :**
   Un client défaillant ou malveillant peut mentir sur son nombre d'échantillons n_k ou envoyer des poids corrompus (attaque par empoisonnement). Le serveur décide alors lui-même du poids de contribution α_k :
   - **Par alignement géométrique (Similarité Cosinus) :** Le serveur compare la direction du vecteur de mise à jour $\Delta w_k^t = w_k^{t+1} - w_t$ d'un client avec la moyenne des autres clients. Si un client dévie fortement (angle opposé), son poids de contribution est réduit ou annulé.

- **Par validation marginale :** Le serveur teste la matrice de chaque client sur un petit jeu de validation de référence et attribue un poids proportionnel au gain de précision réel apporté par ce client.

4 Quels sont les facteurs qui décident du nombre de manches (Rounds) ?

Le nombre total de rounds T dépend de quatre facteurs principaux :

1. **Critère de convergence (Seuil d'arrêt) :** On arrête l'apprentissage lorsque la perte globale $F(w_t)$ stagne sur plusieurs rounds consécutifs ($\|F(w_t) - F(w_{t-1})\| < \epsilon$) ou qu'une précision cible est atteinte.
2. **Le degré d'hétérogénéité (Non-IID) :** Des données IID convergent en quelques rounds (ex: 3 rounds sur notre MNIST partitionné aléatoirement). Des données fortement Non-IID nécessitent des dizaines ou centaines de rounds pour réconcilier les gradients divergents.
3. **Le nombre d'époques locales (E) :** Augmenter le travail local (E) réduit le nombre de rounds nécessaires pour converger, mais au-delà d'un certain seuil, un E trop grand sur des données Non-IID accentue la dérive locale et dégrade la convergence.
4. **Le budget de communication et d'énergie :** En environnement IoT contraint, chaque round a un coût réseau (bande passante) et énergétique. Le nombre de rounds est alors borné par ce budget matériel.

5 Les stratégies d'agrégation serveur alternatives à FedAvg

Pour pallier les limites de la simple moyenne arithmétique pondérée (FedAvg), la littérature propose plusieurs alternatives résumées dans le tableau ci-dessous :

Stratégie	Problème résolu	Principe mathématique et fonctionnement
FedSGD	Dérive locale ($E > 1$)	Les clients calculent un seul gradient sur 1 batch ($E = 1$) et envoient le gradient $\nabla F_k(w_t)$ au serveur au lieu des poids finis.
FedProx	Hétérogénéité (Non-IID)	Ajoute un terme de régularisation proximal $\frac{\mu}{2}\|w - w_t\|^2$ à la fonction de coût locale pour interdire aux poids locaux de trop s'éloigner du modèle global.
FedOpt	Oscillations et lenteur	Le serveur traite la différence $\Delta_t = \bar{w}_{t+1} - w_t$ comme un pseudo-gradient et applique un optimiseur adaptatif (FedAdam, FedYogi) côté serveur.
FedNova	Hétérogénéité matérielle	Normalise les mises à jour locales par le nombre de pas de gradient réellement effectués (τ_k) par chaque client avant d'agréger.
Krum / Median	Attaques Byzantines	Remplace la moyenne par la médiane coordonnée par coordonnée ou sélectionne le modèle local le plus proche de ses voisins pour filtrer les valeurs aberrantes.

6 Peut-on logiquement faire le test sur le même `test_set` ?

La réponse dépend de la distinction entre **évaluation académique (simulation)** et **déploiement industriel réel** :

- **OUI, dans un cadre expérimental de référence (Benchmarking global) :**
  Pour mesurer scientifiquement si le modèle global s'améliore d'un round à l'autre sans biais, il est indispensable d'évaluer le modèle agrégé sur un `test_set` global fixe et équilibré couvrant l'ensemble des distributions (comme effectué dans notre simulation MNIST et PyTorch).
- **NON, dans un déploiement réel strict (Privacy-Preserving & Non-IID) :**
  1. *Argument de confidentialité :* En conditions réelles, le serveur central n'a accès à aucune donnée brute. Construire un `test_set` centralisé impliquerait que les clients envoient une partie de leurs données au serveur, ce qui viole le principe fondamental du Federated Learning.
  2. *Argument statistique (Personnalisation) :* Si l'Usine A possède uniquement des pompes hydrauliques et l'Usine B uniquement des bras robotisés (Non-IID), évaluer les deux clients sur un `test_set` identique n'a pas de sens opérationnel. En pratique (**Federated Evaluation**), chaque client évalue le modèle sur son **propre `test_set` local privé**, puis renvoie uniquement son score scalaire (`loss`, `accuracy`) au serveur qui calcule la moyenne pondérée des métriques.